

PROJECT NAME: MOVIES_RATING_DATASET

Prepare the Final Report

1. Problem statement
2. Proposed solution
3. Tools and technologies
4. Data cleaning
5. Analysis
6. Visualizations
7. Dashboard
8. Key findings
9. Conclusions

1. Define the Problem of the Project

The project aims to analyze a movie rating dataset to understand movie performance and audience preferences.

The main problems to identify are:

- Which movies have the highest ratings?
- Which movies have the most votes?
- What is the average movie rating?
- Which movies can be considered highly rated based on the available data?

2. Propose the Solution

The solution is to clean, analyze, and visualize the movie rating dataset using data analytics tools.

The project will:

- Clean missing and duplicate data.
- Analyze movie ratings and vote counts.
- Calculate average ratings.
- Identify highly rated movies.
- Create charts and visualizations to show the findings.
- Provide useful insights based on the analysis.

3. Decide the Tools and Technologies

The following tools and technologies will be used:

- **Python** – Data cleaning and analysis
- **Pandas** – Data manipulation and analysis
- **Matplotlib** – Data visualization
- **Jupyter Notebook / VS Code** – Writing and executing Python code
- **Excel** – Viewing and preparing the dataset
- **Power BI** – Creating an interactive dashboard

4. Steps to Complete the Project

Step 1: Collect the Dataset

Obtain the Movies Rating Dataset in CSV or Excel format.

Step 2: Load the Dataset

Load the dataset into Python using Pandas.

Step 3: Understand the Dataset

Check:

- Number of rows and columns
- Column names
- Data types
- First few records
- Basic statistical information

```
import pandas as pd
import matplotlib.pyplot as plt
✓ 0.0s

df = pd.read_excel("C:/Users/Test/Desktop/Data_Analytics/Python/Project/Movies_Rating_Dataset_Project2.xlsx")
✓ 1.2s

df.head()
✓ 0.4s
```

	id	original_language	popularity	vote_count	vote_average	release_date	title
0	278	en	36.1467	29384	8.714	1994-09-23	Shark Bait
1	238	en	37.3341	22174	8.686	1972-03-14	Major League II
2	240	en	14.6729	13404	8.571	1974-12-20	Aliens in the Attic
3	424	en	13.6783	16923	8.566	1993-12-15	100 Feet
4	389	en	10.6929	9589	8.548	1957-04-10	The Art of War

```
df.head()
```

✓ 0.4s

	id	original_language	popularity	vote_count	vote_average	release_date	title
0	278	en	36.1467	29384	8.714	1994-09-23	Shark Bait
1	238	en	37.3341	22174	8.686	1972-03-14	Major League II
2	240	en	14.6729	13404	8.571	1974-12-20	Aliens in the Attic
3	424	en	13.6783	16923	8.566	1993-12-15	100 Feet
4	389	en	10.6929	9589	8.548	1957-04-10	The Art of War

```
df.info()
```

✓ 0.5s

```
<class 'pandas.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   id                     10000 non-null  int64  
1   original_language     10000 non-null  str     
2   popularity             10000 non-null  float64 
3   vote_count            10000 non-null  int64  
4   vote_average          10000 non-null  float64 
5   release_date          9998 non-null   datetime64[us]
6   title                 8580 non-null   object  
dtypes: datetime64[us](1), float64(2), int64(2), object(1), str(1)
memory usage: 547.0+ KB
```

```
df.columns
```

✓ 0.0s

```
Index(['id', 'original_language', 'popularity', 'vote_count', 'vote_average',
       'release_date', 'title'],
      dtype='str')
```

Step 4: Clean the Data

- Find and handle missing values.
- Remove duplicate records.
- Correct incorrect data types.
- Check for inconsistent movie

```
# Find missing values
print(df.isnull().sum())
```

✓ 0.0s

```
id                0
original_language 0
popularity        0
vote_count        0
vote_average      0
release_date      2
title            1420
dtype: int64
```

```
# Remove duplicate records.
df = df.dropna()
```

✓ 0.1s

```
# Check again
print(df.isnull().sum())
```

[19] ✓ 0.0s

```
... id                0
original_language    0
popularity           0
vote_count           0
vote_average         0
release_date         0
title               0
dtype: int64
```

```
#Correct incorrect data types.
print(df.dtypes)
```

[20] ✓ 0.0s

```
... id                int64
original_language     str
popularity            float64
vote_count            int64
vote_average          float64
release_date          datetime64[us]
title                 object
dtype: object
```

```
#Check for inconsistent movie names
print("Unique Movie Names:")
print(df["title"].unique())
```

✓ 0.0s

Unique Movie Names:

```
['Shark Bait' 'Major League II' 'Aliens in the Attic' ...
 'Star Trek V: The Final Frontier' 'The Lost Prince' 'Let Us Prey']
```

```
# Remove extra spaces
df["title"] = df["title"].str.strip()
```

✓ 0.0s

```
# Check again
print(df["title"].unique())
```

✓ 0.0s

```
['Shark Bait' 'Major League II' 'Aliens in the Attic' ...
 'Star Trek V: The Final Frontier' 'The Lost Prince' 'Let Us Prey']
```

```
#Title format proper format
df["title"] = df["title"].str.title()

print(df.title)
```

[29] ✓ 0.0s

```
... 0          Shark Bait
     1      Major League Ii
     2    Aliens In The Attic
     3          100 Feet
     4      The Art Of War
     ...
     8575      Bulletproof
     8576      Rough Night
     8577          Haywire
     8578      Prom Night
     8579      The Thaw
     Name: title, Length: 8578, dtype: object
```

Step 5: Analyze the Data

Perform analysis such as:

- Average movie rating
- Highest-rated movies
- Lowest-rated movies
- Most-voted movies

```
#Average movie rating
average_rating = df["vote_average"].mean()

print("Average Movie Rating:", average_rating)

#Two decimal Place
print("Average Movie Rating:", round(df["vote_average"].mean(), 2))
```

✓ 0.0s

Average Movie Rating: 6.88276964327349

Average Movie Rating: 6.88

```
#Highest-rated movies
highest Rated = df.sort_values("vote_average", ascending=False)

print(highest Rated[["title", "vote_average"]].head(10))
```

✓ 0.0s

	title	vote_average
0	Shark Bait	8.714
1	Major League II	8.686
2	Aliens In The Attic	8.571
3	100 Feet	8.566
4	The Art Of War	8.548
5	Nine Months	8.533
6	Timeline	8.524
8	Skiptrace	8.503
7	Redcon-1	8.500
9	47 Meters Down: Uncaged	8.496

```
#Lowest-rated movies
lowest Rated = df.sort_values("vote_average", ascending=True)

print(lowestRated[["title", "vote_average"]].head(10))
```

✓ 0.0s

	title	vote_average
8572	Billionaire Boys Club	5.900
8571	The Getaway	5.900
8579	The Thaw	5.947
8577	Haywire	5.947
8578	Prom Night	5.947
8576	Rough Night	5.948
8573	Tarzan	5.948
8574	Quarantine 2: Terminal	5.948
8575	Bulletproof	5.948
8570	Hall Pass	5.949

```
#Mosted - Voted Movies
most_voted = df.sort_values("vote_count", ascending=False)

print(most_voted[["title", "vote_count"]].head(10))
```

✓ 0.1s

	title	vote_count
14	Quarantine 2: Terminal	38436
41	Interstellar	38375
6	Timeline	34849
465	Finch	34558
1038	Spies In Disguise	32998
964	A Streetcar Named Desire	32153
93	Le Trou	31232
20	The Shawshank Redemption	31114
11	The Getaway	29404
0	Shark Bait	29384

Step 6: Create Visualizations

Create charts such as:

- Bar chart for top-rated movies

- Bar chart for popularity ratings
- Histogram for movie ratings
- Scatter plot for ratings and number of votes

Bar chart for top-rated movies:

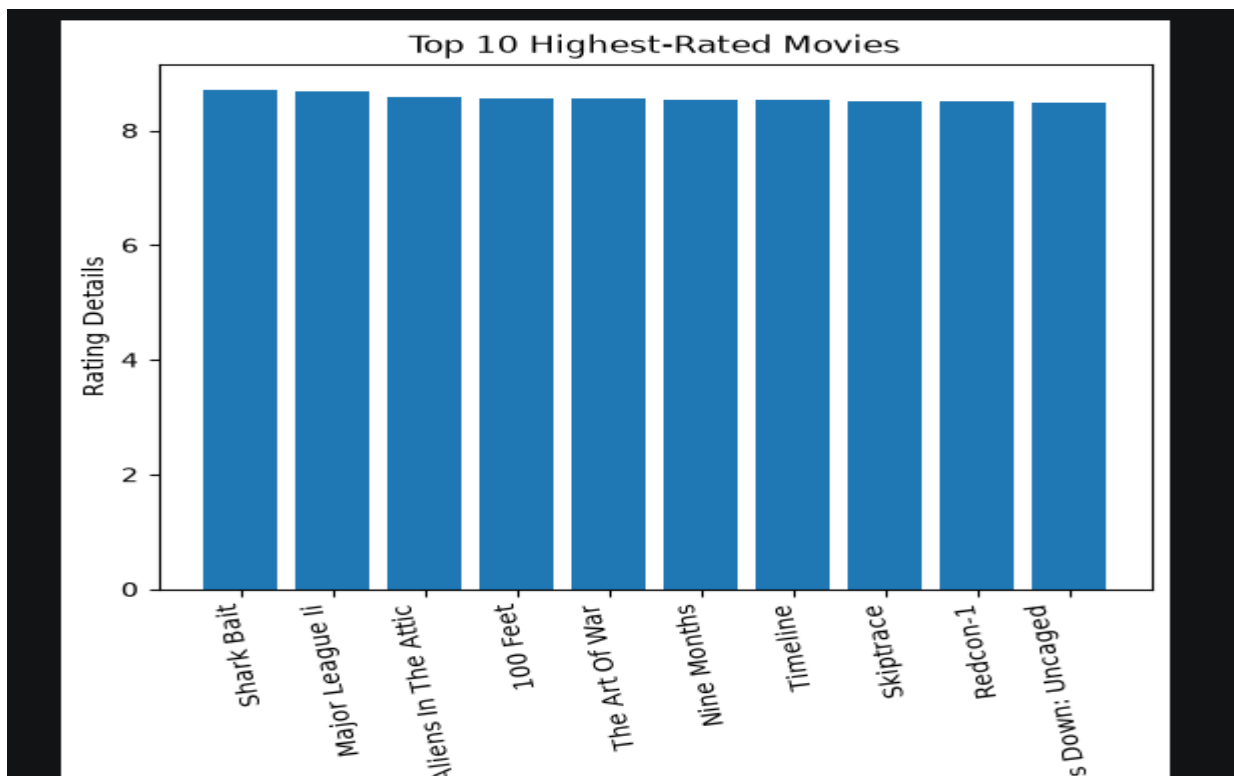
```
import matplotlib.pyplot as plt

top Rated = df.sort_values("vote_average", ascending=False).head(10)

plt.bar(top Rated["title"], top Rated["vote_average"])

plt.xlabel("Movie Title")
plt.ylabel("Rating Details")

plt.title("Top 10 Highest-Rated Movies")
plt.xticks(rotation=100)
plt.show()
```



Bar chart for Popularity ratings:

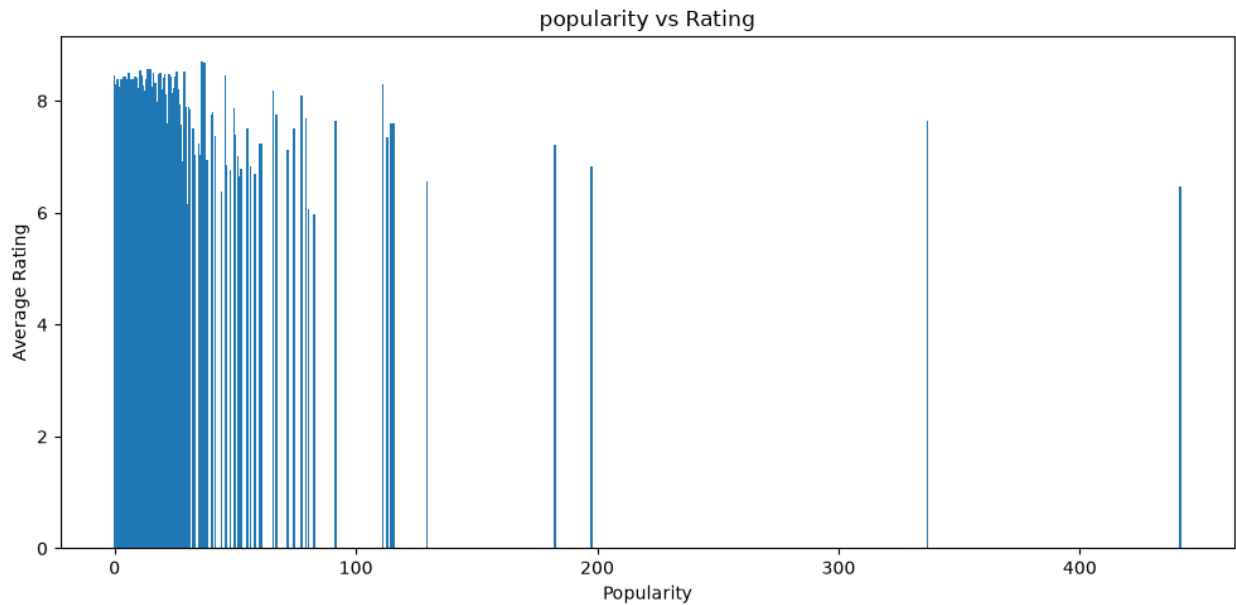
```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(10, 5))

plt.bar(df["popularity"],df["vote_average"])

plt.xlabel("Popularity")
plt.ylabel("Average Rating")
plt.title("popularity vs Rating")

plt.tight_layout()
plt.show()
```



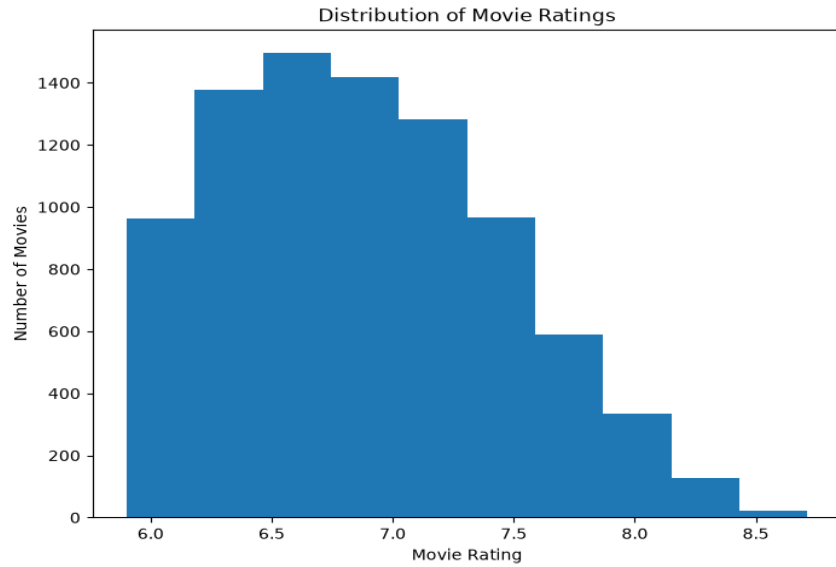
Histogram for movie ratings:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 6))
plt.hist(df["vote_average"], bins=10)

plt.xlabel("Movie Rating")
plt.ylabel("Number of Movies")

plt.title("Distribution of Movie Ratings")
```



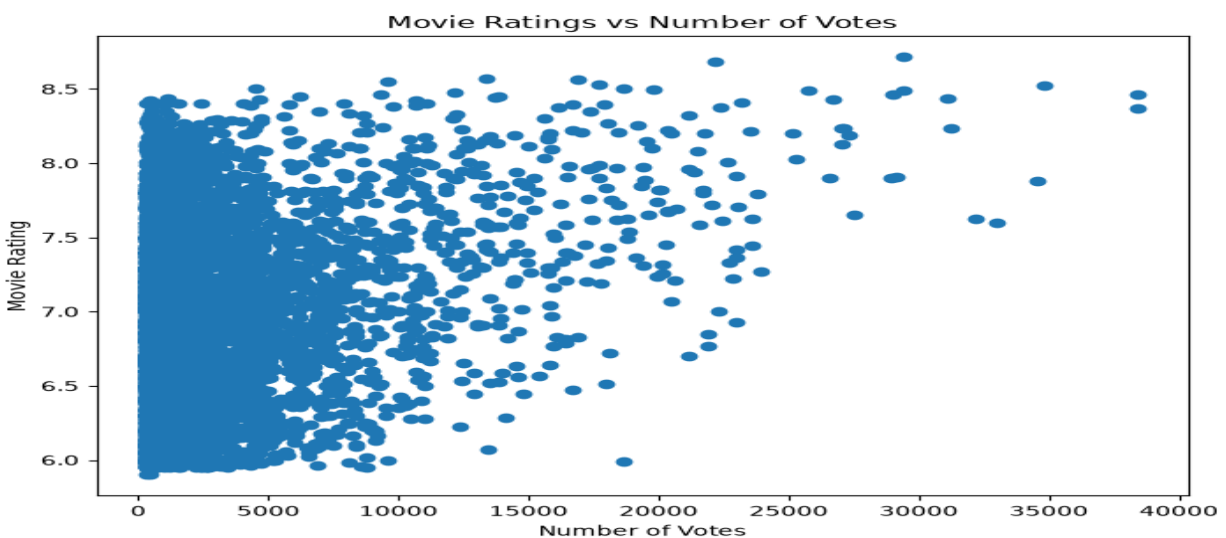
Scatter plot for ratings and number of votes:

```
import matplotlib.pyplot as plt
plt.figure(figsize=(8, 6))

plt.scatter(df["vote_count"], df["vote_average"])

plt.xlabel("Number of Votes")
plt.ylabel("Movie Rating")
plt.title("Movie Ratings vs Number of Votes")

plt.tight_layout()
plt.show()
```

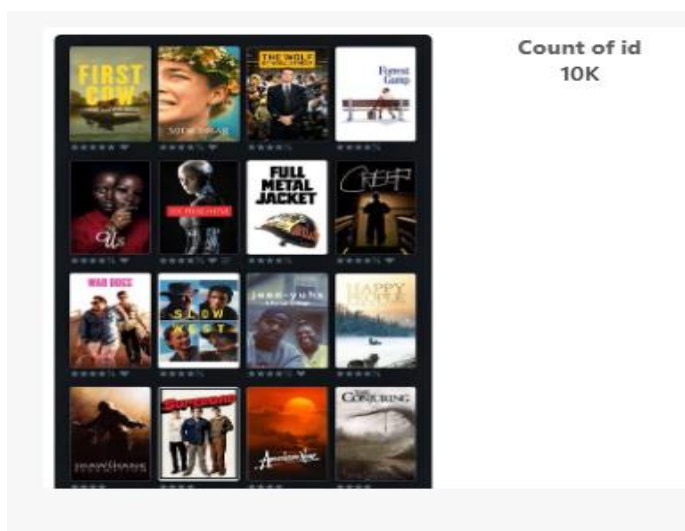


Step 7: Create a Dashboard

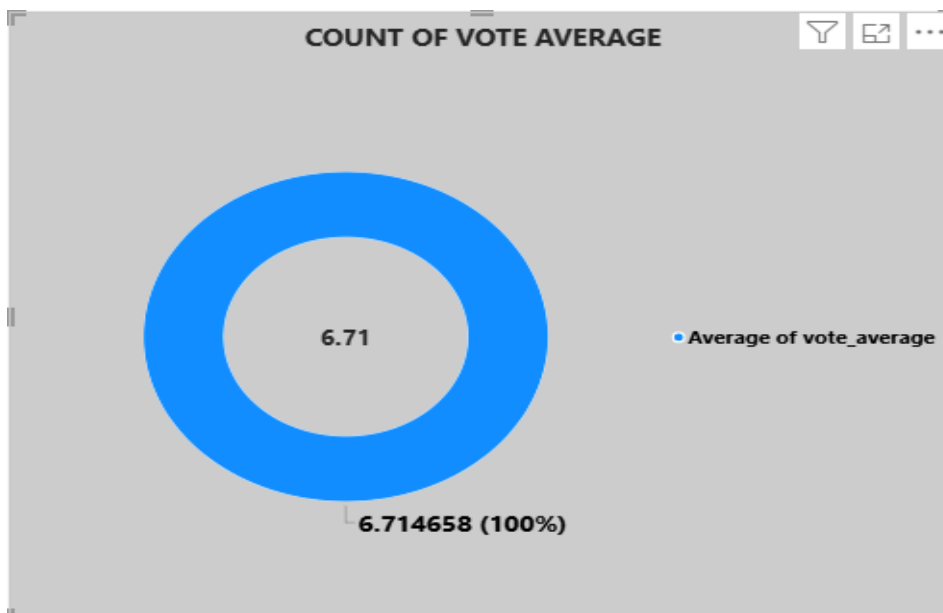
Use Power BI to create an interactive dashboard containing:

- Total number of movies
- Average rating
- Highest rating
- Top-rated movies
- Rating distribution

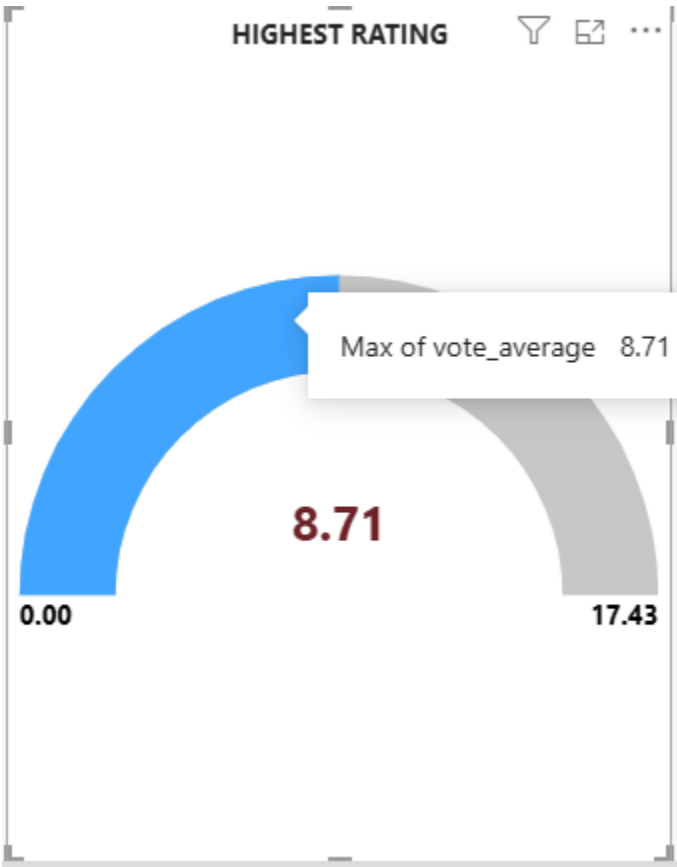
Total number of movies



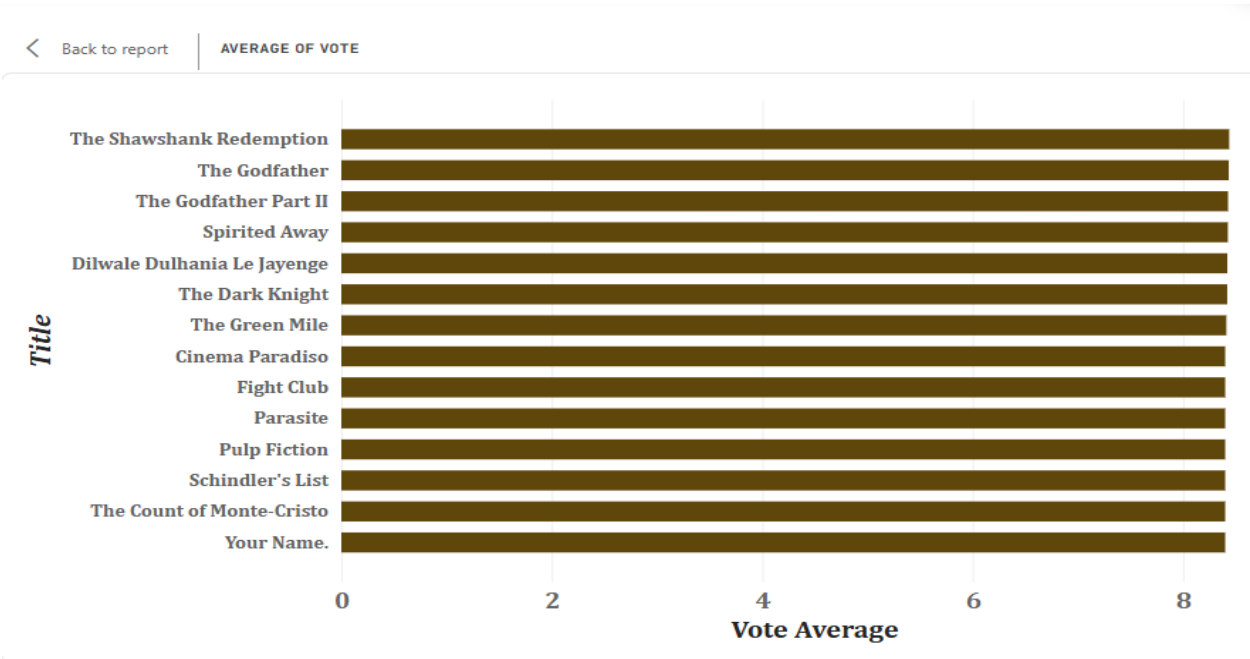
Average rating:



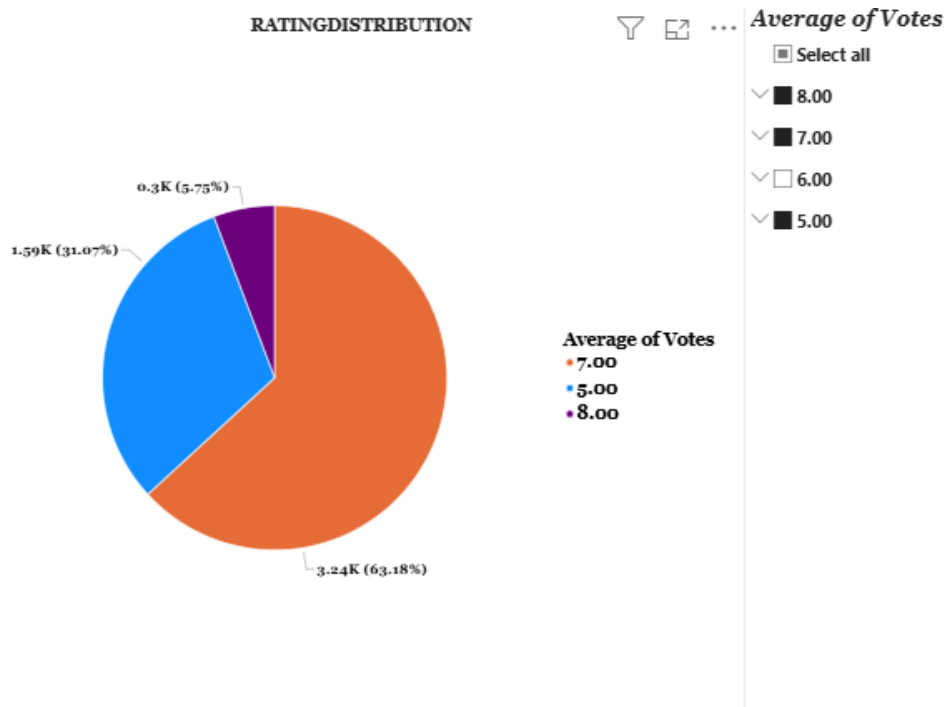
Highest rating:



Top-rated movies:



Rating distribution:



Step 8: Find Insights

Identify important findings from the analysis, highly rated movies, and relationships between ratings and votes.

Key Findings:

- The analysis identified the highest-rated movies in the dataset.
- The rating distribution showed the different rating levels of the movies.
- The analysis compared movie popularity with ratings.
- The scatter plot showed the relationship between the number of votes and movie ratings.
- The Power BI dashboard provided an interactive view of movie ratings and related information.

Step 8: Conclusion

The movie rating dataset was successfully analyzed using Python and Power BI. The data was cleaned and analyzed to understand movie ratings, popularity, vote counts, and language distribution. Visualizations and an interactive dashboard helped to identify important patterns in the dataset. Overall, the project provided useful insights into movie ratings, popularity, and voting patterns.